In [1]:
```
#Nama: Alya Fitriani
#Nim: TI22250004
#data understanding
```

In [2]:
```
import pandas as pd
import numpy as np
```

In [3]:
```
df = pd.read_csv("original.csv")
df.head() #menampilkan 5 data pertama
```

Out[3]:

	Gender	Age	POLAR4 Quintile	POLAR3 Quintile	Adult HE 2001 Quintile	Adult HE 2011 Quintile	TUNDRA MSOA Quintile	TUNDRA LSOA Quintile	Gaps GCSE Quintile	E
0	0.0	38.0	5.0	5.0	4.0	5.0	4.0	4.0	5.0	
1	0.0	22.0	4.0	2.0	1.0	3.0	4.0	4.0	4.0	
2	0.0	21.0	4.0	5.0	4.0	5.0	4.0	4.0	5.0	
3	0.0	25.0	5.0	4.0	3.0	4.0	3.0	3.0	5.0	
4	0.0	20.0	5.0	4.0	3.0	4.0	3.0	3.0	4.0	

5 rows × 22 columns

In [4]:
```
#jumlah ukuran kolom dan baris dataset/dimensinya
print("Ukuran Dataset:", df.shape)
```

```
Ukuran Dataset: (87, 22)
```

In [5]:
```
#Dataset memiliki 87 baris dan 22 kolom
```

In [6]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 87 entries, 0 to 86
Data columns (total 22 columns):
 #   Column                              Non-Null Count  Dtype
---  ------                              --------------  -----
 0   Gender                              80 non-null     float64
 1   Age                                 80 non-null     float64
 2   POLAR4 Quintile                     80 non-null     float64
 3   POLAR3 Quintile                     80 non-null     float64
 4   Adult HE 2001 Quintile              80 non-null     float64
 5   Adult HE 2011 Quintile              80 non-null     float64
 6   TUNDRA MSOA Quintile                80 non-null     float64
 7   TUNDRA LSOA Quintile                80 non-null     float64
 8   Gaps GCSE Quintile                  80 non-null     float64
 9   Gaps GCSE Ethnicity Quintile        80 non-null     float64
 10  Uni Connect target ward             80 non-null     float64
 11  attending from home?                80 non-null     float64
 12  distance to university (km)         80 non-null     float64
 13  Count of Module Area Logins         80 non-null     float64
 14  Total Hours in Module Area          80 non-null     float64
 15  % of Average Hours in Module Area   80 non-null     float64
 16  # of presence                       80 non-null     float64
 17  # of Absence                        78 non-null     float64
 18  Percent Attended                    80 non-null     float64
 19  label (fail=1, pass=0)              80 non-null     float64
 20  Unnamed: 20                         0 non-null      float64
 21  Unnamed: 21                         0 non-null      float64
dtypes: float64(22)
memory usage: 15.1 KB
```

In [7]:
```
#Sebagian kolom memilki 80 data terisi
#kolom #of Absence 78 data terisi (2 nilai kosong pada kolom #of Absence
#Tipe data float64
```

kelompok fitur: -3 kelompok fitur utama

1. Demografi (Gender, Age) mencakup jenis kelamin dan usia mahasiswa.
2. Latar belakang wilayah (POLAR4, POLAR3, Adult HE, TUNDRA, Gaps GCSE, Uni Connect) menggambarkan indikator partisipasi pendidikan tinggi dan kondisi pendidikan di wilayah asal mahasiswa.
3. Keterlibatan belajar (Attending from home, Distance, Logins, Total Hours, Presence, Absence, Percent Attended) keterlibatan belajar yang mencakup aktivitas mahasiswa dalam menggunakan modul perkuliahan, waktu belajar, jarak ke universitas, serta tingkat kehadiran.

In [8]:
```
#Menentukan fitur (X) dan label (y)
```

Fitur (X) -> seluruh variabel yang digunakan senagai dasar prediksi. Label (y) -> hasil yang ingin di prediksi. Kolom label (fail=1, pass=0) ditetapkan sebagai label karena menunjukkan status kelulusan mahasiswa, dengan nilai 0 berarti lulus dan nilai 1 berarti gagal.

In [9]:
```
#Distribusi label
df["label (fail=1, pass=0)"].value_counts()
```

Out[9]:
```
label (fail=1, pass=0)
0.0    68
1.0    12
Name: count, dtype: int64
```

In [10]:
```
df['label (fail=1, pass=0)'].value_counts(normalize=True) * 100
```

Out[10]:
```
label (fail=1, pass=0)
0.0    85.0
1.0    15.0
Name: proportion, dtype: float64
```

Berdasarkan hasil distribusi label, mahasiswa yang lulus 68 (85%) dan mahasiswa yang gagal 12 (15%). Ini menunjukkan bahwa dataset memiliki distribusi kelas yang tidak seimbang (imbalanced data), karena jumlah mahasiswa yang lulus jauh lebih banyak dibandingkan mahasiswa yang gagal. Ketidakseimbangan ini perlu diperhatikan dalam proses pembuatan model. Jika hanya menggunakan akurasi sebagai ukuran keberhasilan, model dapat terlihat memiliki hasil yang baik meskipun kurang mampu mengenali mahasiswa yang gagal.

In [11]:
```
#EDA dan Pemprosesan Data
```

In [12]:
```
#Baris kosong
baris_kosong = df.isnull().all(axis=1).sum()
print("Jumlah baris kosong:",baris_kosong)
```

```
Jumlah baris kosong: 7
```

In [13]:
```
kolom_kosong = df.isnull().all(axis=0).sum()
print("Jumlah kolom kosong:", kolom_kosong)
```

```
Jumlah kolom kosong: 2
```

In [14]:
```
#missing value (jumlah nilai kosong)
df.isnull().sum()
```

Out[14]:
```
Gender                                 7
Age                                    7
POLAR4 Quintile                        7
POLAR3 Quintile                        7
Adult HE 2001 Quintile                 7
Adult HE 2011 Quintile                 7
TUNDRA MSOA Quintile                   7
TUNDRA LSOA Quintile                   7
Gaps GCSE Quintile                     7
Gaps GCSE Ethnicity Quintile           7
Uni Connect target ward                7
attending from home?                   7
distance to university (km)            7
Count of Module Area Logins            7
Total Hours in Module Area             7
% of Average Hours in Module Area      7
# of presence                          7
# of Absence                           9
Percent Attended                       7
label (fail=1, pass=0)                 7
Unnamed: 20                           87
Unnamed: 21                           87
dtype: int64
```

In [15]:
```
#duplikat
print("Jumlah data duplikat:", df.duplicated().sum())
```

```
Jumlah data duplikat: 6
```

berdasarkan hasil df.insull().sum() menunjukkan baris kosong ada 7, kolom kosong 2, nilai kosong 9, dan duplikat 6.

In [16]:
```
#pembersihan baris kosong
df=df.dropna(how='all')
print("Ukuran dataset:", df.shape)
```

```
Ukuran dataset: (80, 22)
```

Menghapus 7 baris kosong yang seluruh kolomnya kosong karena tidak memiliki informasi mahasiswa yang dapat digunakan dalam analisis.

In [17]:
```
#menghapus kolom kosong
df=df.dropna(axis=1, how='all')
print("Ukuran dataset:", df.shape)
```

```
Ukuran dataset: (80, 20)
```

menghapus 2 kolom yang tidak memiliki informasi yang berguna

In [18]:
```
#memeriksa kembali duplikat
print("Jumlah duplikat:", df.duplicated().sum())
```

```
Jumlah duplikat: 0
```

In [19]:
```
#menangani missing values
```

In [20]:
```
#memeriksa kembali nilai kosong
df.isnull().sum()
```

Out[20]:
```
Gender                                  0
Age                                     0
POLAR4 Quintile                         0
POLAR3 Quintile                         0
Adult HE 2001 Quintile                  0
Adult HE 2011 Quintile                  0
TUNDRA MSOA Quintile                    0
TUNDRA LSOA Quintile                    0
Gaps GCSE Quintile                      0
Gaps GCSE Ethnicity Quintile            0
Uni Connect target ward                 0
attending from home?                    0
distance to university (km)             0
Count of Module Area Logins             0
Total Hours in Module Area              0
% of Average Hours in Module Area       0
# of presence                           0
# of Absence                            2
Percent Attended                        0
label (fail=1, pass=0)                  0
dtype: int64
```

In [21]:
```
#isi missing value dengan median
median_absence = df['# of Absence'].median()
df['# of Absence'] = df['# of Absence'].fillna(median_absence)
print("Median Absence:", median_absence)
```

```
Median Absence: 2.0
```

Median yaitu nilai tengah data. Menggunakan metode ini karena lebih tahan terhadap pengaruh nilai ekstrem (outlier) dibandingkan rata-rata.

In [22]:
```
#memeriksa kembali pembersihan
print(df.isnull().sum())
```

```
Gender                                  0
Age                                     0
POLAR4 Quintile                         0
POLAR3 Quintile                         0
Adult HE 2001 Quintile                  0
Adult HE 2011 Quintile                  0
TUNDRA MSOA Quintile                    0
TUNDRA LSOA Quintile                    0
Gaps GCSE Quintile                      0
Gaps GCSE Ethnicity Quintile            0
Uni Connect target ward                 0
attending from home?                    0
distance to university (km)             0
Count of Module Area Logins             0
Total Hours in Module Area              0
% of Average Hours in Module Area       0
# of presence                           0
# of Absence                            0
Percent Attended                        0
label (fail=1, pass=0)                  0
dtype: int64
```

Ditemukan dua nilai kosong pada # of Absence setelah baris kosong dihapus. Nilai kosong ditangani menggunakan median karena median tidak mudah dipengaruhi oleh

nilai ekstrem. Setelah pengisian, pemeriksaan ulang dilakukan untuk memastikan tidak ada missing values lagi.

In [23]:
```
df.columns = df.columns.str.strip()
```

In [24]:
```
#Memeriksa jumlah nilai 0 pada kolom kuintil
kolom_kuintil = [
    'POLAR4 Quintile',
    'POLAR3 Quintile',
    'Adult HE 2001 Quintile',
    'Adult HE 2011 Quintile',
    'TUNDRA MSOA Quintile',
    'TUNDRA LSOA Quintile',
    'Gaps GCSE Quintile',
    'Gaps GCSE Ethnicity Quintile'
]

for kolom in kolom_kuintil:
    print(kolom, ":", (df[kolom] == 0).sum())
```

```
POLAR4 Quintile : 0
POLAR3 Quintile : 0
Adult HE 2001 Quintile : 0
Adult HE 2011 Quintile : 2
TUNDRA MSOA Quintile : 2
TUNDRA LSOA Quintile : 2
Gaps GCSE Quintile : 4
Gaps GCSE Ethnicity Quintile : 4
```

Pemeriksaan ini perlu dilakukan karena nilai 0 tidak termasuk rentang kuintil normal (1-5). Kita harus tahu kolom mana yang memiliki nilai tersebut sebelum menentukan penanganan. Terdapat 14 nilai 0 pada 5 kolom kuintil. Berdasarkan keterangan dataset, nilai kuintil normal berada pada rentang 1-5. Oleh karena itu, nilai 0 perlu diperiksa karena berada di luar rentang.

In [25]:
```
# Menampilkan data yang memiliki nilai 0 pada kolom kuintil

for kolom in kolom_kuintil:
    data_nol = df[df[kolom] == 0]

    if not data_nol.empty:
        print("\nKolom:", kolom)
        display(data_nol[[kolom, 'label (fail=1, pass=0)']])
```

```
Kolom: Adult HE 2011 Quintile
```

	Adult HE 2011 Quintile	label (fail=1, pass=0)
23	0.0	0.0
66	0.0	0.0

```
Kolom: TUNDRA MSOA Quintile
```

	TUNDRA MSOA Quintile	label (fail=1, pass=0)
35	0.0	0.0
74	0.0	0.0

Kolom: TUNDRA LSOA Quintile

	TUNDRA LSOA Quintile	label (fail=1, pass=0)
35	0.0	0.0
74	0.0	0.0

Kolom: Gaps GCSE Quintile

	Gaps GCSE Quintile	label (fail=1, pass=0)
23	0.0	0.0
35	0.0	0.0
66	0.0	0.0
74	0.0	0.0

Kolom: Gaps GCSE Ethnicity Quintile

	Gaps GCSE Ethnicity Quintile	label (fail=1, pass=0)
23	0.0	0.0
35	0.0	0.0
66	0.0	0.0
74	0.0	0.0

Tujuan pemeriksaan:

1. Mengetahui baris mahasiswa yang memiliki nilai 0.
2. Melihat apakah nilai 0 muncul pada mahasiswa lulus atau gagal.
3. Menjadi dasar untuk menentukan penanganan nilai tersebut.

In [26]:
```
import matplotlib.pyplot as plt
import seaborn as sns
```

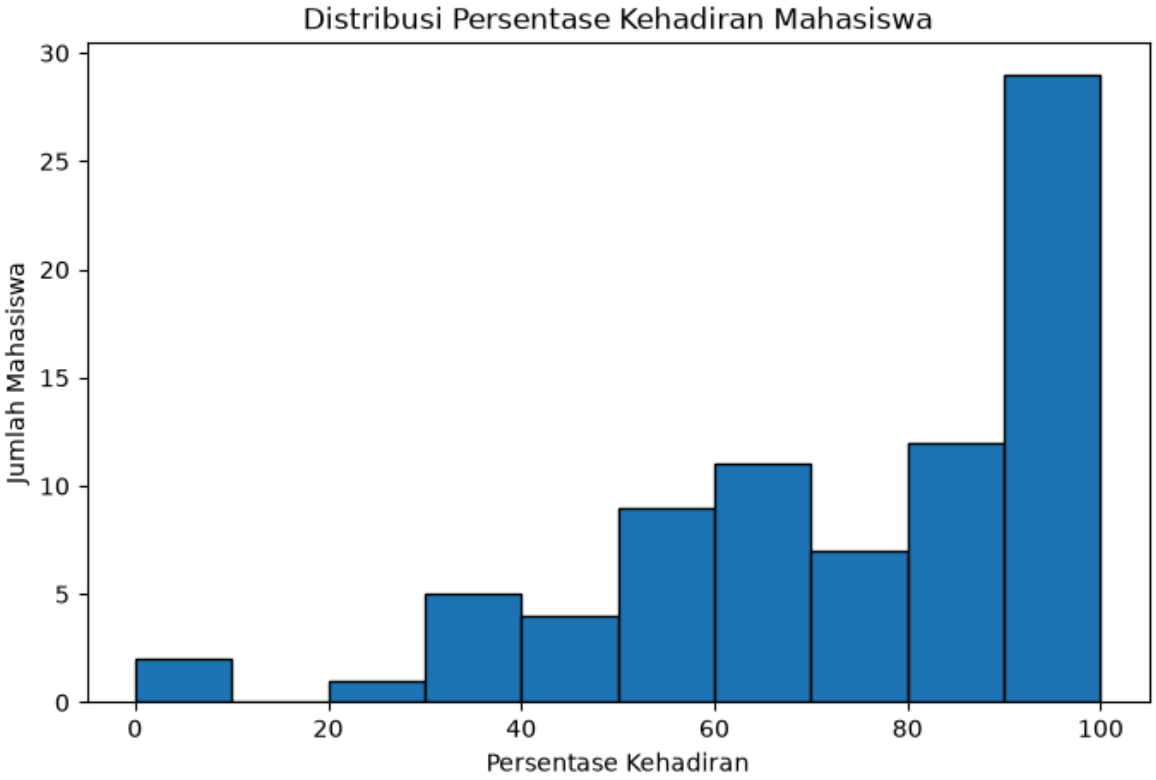
In [27]:
```
# Histogram persentase kehadiran mahasiswa

plt.figure(figsize=(8, 5))

plt.hist(df['Percent Attended'], bins=10, edgecolor='black')

plt.title('Distribusi Persentase Kehadiran Mahasiswa')
plt.xlabel('Persentase Kehadiran')
plt.ylabel('Jumlah Mahasiswa')

plt.show()
```



Berdasarkan histogram, sebagian besar mahasiswa memiliki persentase kehadiran yang tinggi, terutama pada rentang 90–100%. Sementara itu, jumlah mahasiswa dengan persentase kehadiran rendah relatif sedikit. Hal ini menunjukkan bahwa secara umum tingkat kehadiran mahasiswa cukup tinggi, meskipun terdapat beberapa mahasiswa dengan tingkat kehadiran rendah.

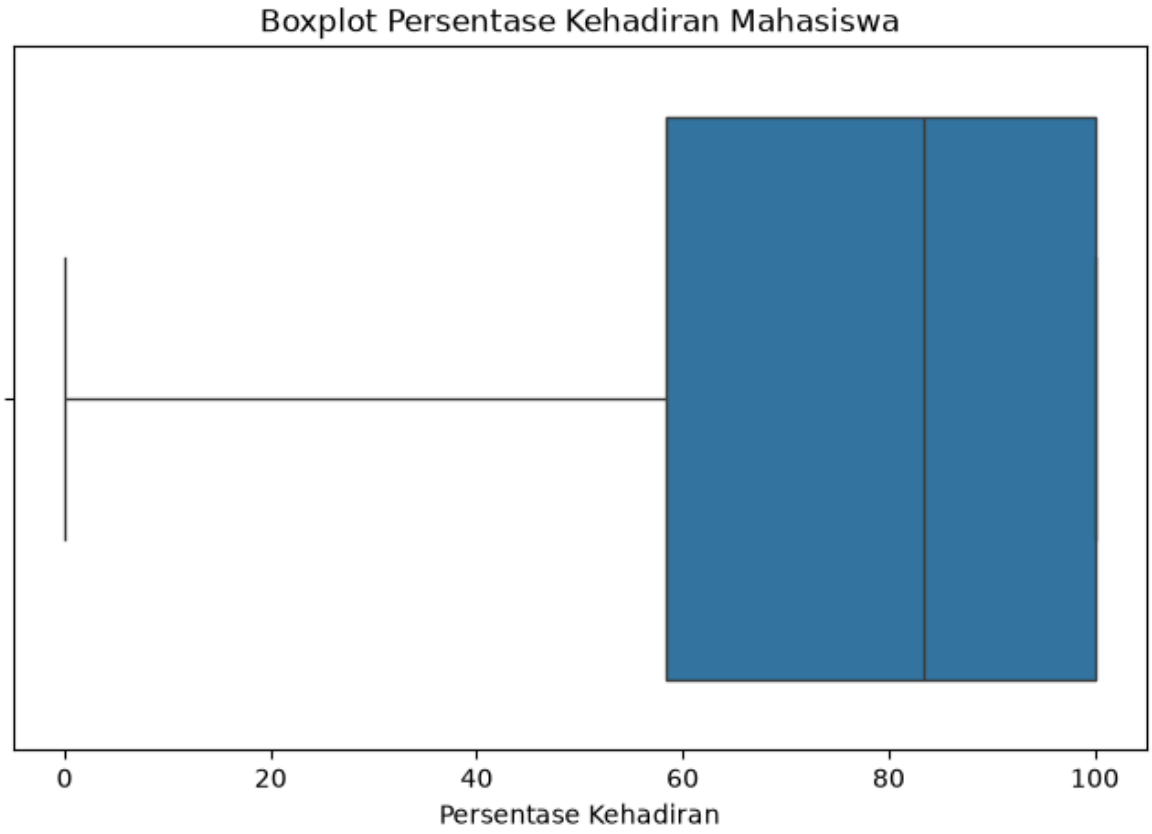
In [28]:
```
# Boxplot persentase kehadiran mahasiswa

plt.figure(figsize=(8, 5))

sns.boxplot(x=df['Percent Attended'])

plt.title('Boxplot Persentase Kehadiran Mahasiswa')
plt.xlabel('Persentase Kehadiran')

plt.show()
```



Berdasarkan hasilnya, persentase kehadiran mahasiswa memiliki penyebaran nilai dari sekitar 0% hingga 100%. Nilai median berada di sekitar 84%, yang menunjukkan bahwa sebagian besar mahasiswa memiliki tingkat kehadiran yang relatif tinggi.Rentang kotak menunjukkan adanya variasi persentase kehadiran antarmahasiswa. Tidak terlihat titik pencilan yang terpisah pada visualisasi tersebut.

In [29]:
```
# Memisahkan data berdasarkan label
lulus = df[df['label (fail=1, pass=0)'] == 0]
gagal = df[df['label (fail=1, pass=0)'] == 1]

# Membuat visualisasi perbandingan
fig, axes = plt.subplots(1, 3, figsize=(16, 5))

# Perbandingan persentase kehadiran
axes[0].boxplot(
    [lulus['Percent Attended'], gagal['Percent Attended']],
    label=['Lulus', 'Gagal']
)
axes[0].set_title('Perbandingan Persentase Kehadiran')
axes[0].set_ylabel('Persentase Kehadiran (%)')

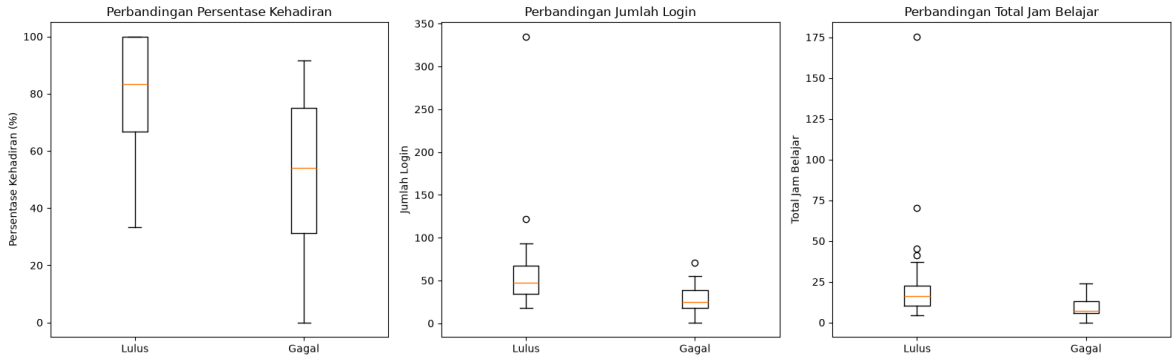
# Perbandingan jumlah login
axes[1].boxplot(
    [lulus['Count of Module Area Logins'],
     gagal['Count of Module Area Logins']],
    label=['Lulus', 'Gagal']
)
axes[1].set_title('Perbandingan Jumlah Login')
axes[1].set_ylabel('Jumlah Login')

# Perbandingan total jam belajar
axes[2].boxplot(
```

```
    [lulus['Total Hours in Module Area'],
     gagal['Total Hours in Module Area']],
    tick_labels=['Lulus', 'Gagal']
)
axes[2].set_title('Perbandingan Total Jam Belajar')
axes[2].set_ylabel('Total Jam Belajar')

axes[0].set_xticks([1, 2], ['Lulus', 'Gagal'])
axes[1].set_xticks([1, 2], ['Lulus', 'Gagal'])

plt.tight_layout()
plt.show()
```



Pada fitur persentase kehadiran, mahasiswa yang lulus memiliki median kehadiran lebih tinggi dibandingkan mahasiswa yang gagal. Hal ini menunjukkan bahwa mahasiswa yang lulus cenderung memiliki tingkat kehadiran yang lebih baik.

Pada fitur jumlah login, persebaran data mahasiswa lulus dan gagal terlihat relatif berdekatan. Namun, terdapat beberapa nilai pencilan (outlier), terutama pada kelompok mahasiswa lulus.

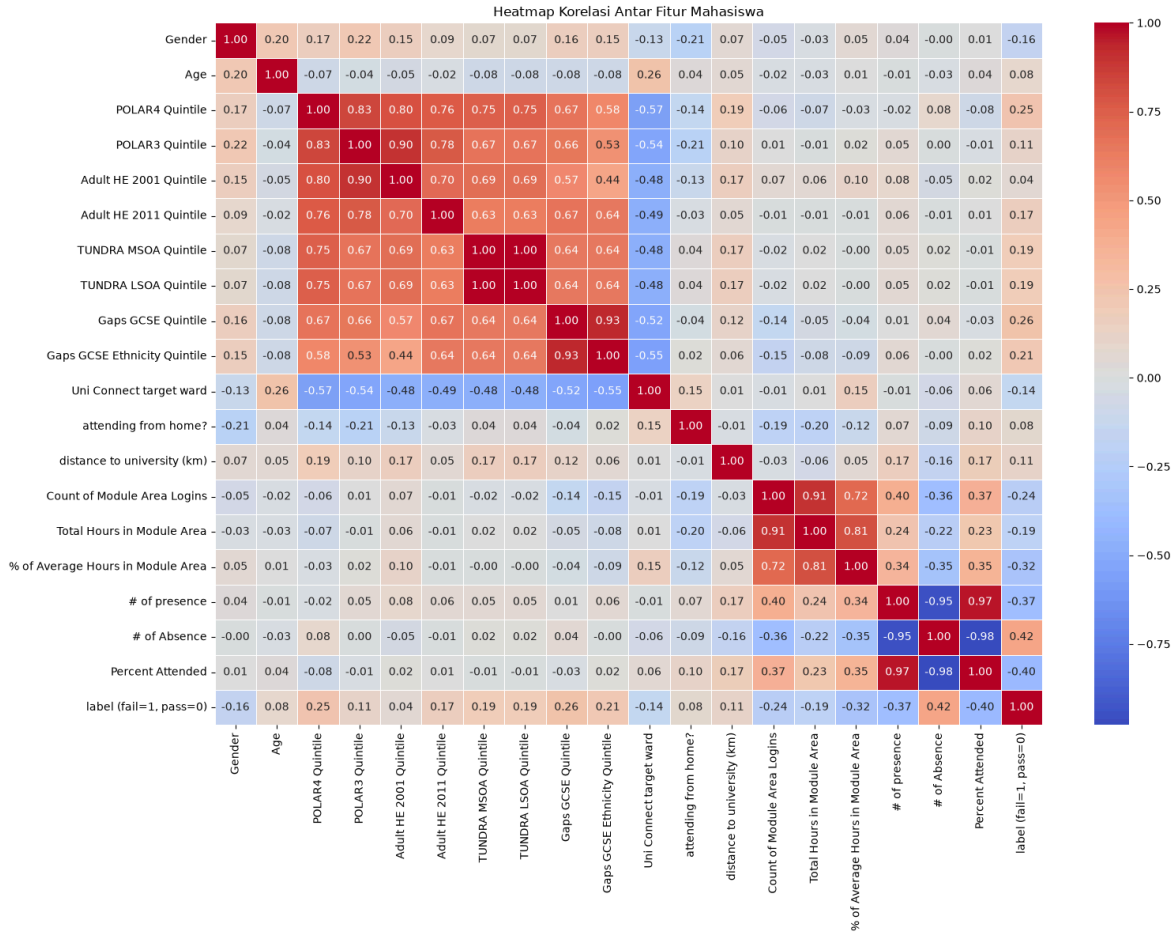
Sementara itu, pada fitur total jam belajar, kedua kelompok memiliki persebaran yang relatif berdekatan. Median mahasiswa gagal terlihat sedikit lebih tinggi dibandingkan mahasiswa lulus. Hal ini menunjukkan bahwa total jam belajar tidak selalu menentukan hasil kelulusan mahasiswa. Secara keseluruhan, persentase kehadiran memperlihatkan perbedaan yang lebih jelas antara mahasiswa lulus dan gagal dibandingkan jumlah login maupun total jam belajar.

In [30]:
```
# Menghitung korelasi antar kolom numerik
correlation_matrix = df.corr(numeric_only=True)

# Membuat visualisasi heatmap
plt.figure(figsize=(16, 12))

sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap='coolwarm',
    fmt='.2f',
    linewidths=0.5
)

plt.title('Heatmap Korelasi Antar Fitur Mahasiswa')
plt.tight_layout()
plt.show()
```



Berdasarkan hasil visualisasi heatmap korelasi, diketahui bahwa terdapat beberapa fitur yang memiliki hubungan positif kuat maupun negatif dalam dataset mahasiswa.

Korelasi positif tertinggi terlihat pada fitur TUNDRA MSOA Quintile dan TUNDRA LSOA Quintile dengan nilai 1,00. Selain itu, fitur Gaps GCSE Quintile dan Gaps GCSE Ethnicity Quintile memiliki korelasi sebesar 0,93, sedangkan POLAR3 Quintile dan Adult HE 2001 Quintile memiliki korelasi sebesar 0,90.

Sementara itu, beberapa fitur memiliki korelasi negatif, seperti Uni Connect target ward dengan POLAR4 Quintile sebesar -0,57. Terdapat pula fitur yang memiliki korelasi lemah karena nilai korelasinya mendekati nol.

Hasil heatmap ini membantu mengetahui hubungan antarfitur serta mengidentifikasi fitur yang memiliki korelasi tinggi. Informasi tersebut dapat digunakan sebagai bahan pertimbangan dalam proses seleksi fitur sebelum membangun model Machine Learning.

In [31]:
```
#Deteksi outlier dan nilai tidak wajar
#mendeteksi outlier pada fitur numerik
df.columns = df.columns.str.strip()

fitur_numerik = [
    'Age',
    'distance to university (km)',
    'Count of Module Area Logins',
    'Total Hours in Module Area',
    '% of Average Hours in Module Area',
    '# of presence',
    '# of Absence',
```

```
    'Percent Attended'
]

# Menghitung outlier menggunakan metode IQR
hasil_outlier = []

for kolom in fitur_numerik:
    Q1 = df[kolom].quantile(0.25)
    Q3 = df[kolom].quantile(0.75)
    IQR = Q3 - Q1

    batas_bawah = Q1 - 1.5 * IQR
    batas_atas = Q3 + 1.5 * IQR

    jumlah_outlier = (
        (df[kolom] < batas_bawah) |
        (df[kolom] > batas_atas)
    ).sum()

    hasil_outlier.append({
        'Fitur': kolom,
        'Q1': Q1,
        'Q3': Q3,
        'Batas Bawah': batas_bawah,
        'Batas Atas': batas_atas,
        'Jumlah Outlier': jumlah_outlier
    })

tabel_outlier = pd.DataFrame(hasil_outlier)

display(tabel_outlier.round(2))
```

	Fitur	Q1	Q3	Batas Bawah	Batas Atas	Jumlah Outlier
0	Age	20.00	22.00	17.00	25.00	6
1	distance to university (km)	3.26	32.45	-40.53	76.24	7
2	Count of Module Area Logins	30.75	62.00	-16.12	108.88	2
3	Total Hours in Module Area	8.55	22.24	-11.98	42.77	3
4	% of Average Hours in Module Area	0.58	1.29	-0.47	2.34	4
5	# of presence	7.00	11.00	1.00	17.00	2
6	# of Absence	0.75	4.25	-4.50	9.50	2
7	Percent Attended	58.33	100.00	-4.17	162.50	0

In [32]:
```
# Memeriksa nilai 0 pada kolom kuintil
kolom_kuintil = [
    kolom for kolom in df.columns
    if 'Quintile' in kolom
]

hasil_nilai_0 = []

for kolom in kolom_kuintil:
```

```
    jumlah_nol = (df[kolom] == 0).sum()

    hasil_nilai_0.append({
        'Nama Kolom': kolom,
        'Jumlah Nilai 0': jumlah_nol
    })

tabel_nilai_0 = pd.DataFrame(hasil_nilai_0)

display(tabel_nilai_0)
```

	Nama Kolom	Jumlah Nilai 0
0	POLAR4 Quintile	0
1	POLAR3 Quintile	0
2	Adult HE 2001 Quintile	0
3	Adult HE 2011 Quintile	2
4	TUNDRA MSOA Quintile	2
5	TUNDRA LSOA Quintile	2
6	Gaps GCSE Quintile	4
7	Gaps GCSE Ethnicity Quintile	4

Berdasarkan hasil deteksi menggunakan metode Interquartile Range (IQR), ditemukan outlier pada beberapa fitur numerik dalam dataset. Jumlah outlier terbanyak terdapat pada fitur distance to university (km), yaitu sebanyak 7 data, diikuti Age sebanyak 6 data, dan % of Average Hours in Module Area sebanyak 4 data.

Sementara itu, fitur Percent Attended tidak menunjukkan adanya outlier berdasarkan metode IQR. Pada pemeriksaan nilai 0 di kolom kuintil, ditemukan nilai 0 pada beberapa fitur, yaitu Adult HE 2011 Quintile sebanyak 2 data, TUNDRA MSOA Quintile sebanyak 2 data, TUNDRA LSOA Quintile sebanyak 2 data, Gaps GCSE Quintile sebanyak 4 data, dan Gaps GCSE Ethnicity Quintile sebanyak 4 data.

Nilai 0 tersebut perlu diperiksa lebih lanjut karena berada di luar rentang kuintil 1–5 apabila mengikuti skala yang dijelaskan dalam dokumentasi dataset.Oleh karena itu, outlier dan nilai 0 tidak langsung dihapus, tetapi perlu diperiksa terlebih dahulu untuk menentukan apakah merupakan nilai yang valid, kesalahan pencatatan, atau penanda data yang tidak tersedia.

In [33]:
```
#memeriksa data yang terdeteksi
# Memeriksa data outlier pada fitur jarak universitas
kolom = 'distance to university (km)'

Q1 = df[kolom].quantile(0.25)
Q3 = df[kolom].quantile(0.75)
IQR = Q3 - Q1

batas_bawah = Q1 - 1.5 * IQR
batas_atas = Q3 + 1.5 * IQR
```

```
data_outlier = df[
    (df[kolom] < batas_bawah) |
    (df[kolom] > batas_atas)
]

display(data_outlier)
```

	Gender	Age	POLAR4 Quintile	POLAR3 Quintile	Adult HE 2001 Quintile	Adult HE 2011 Quintile	TUNDRA MSOA Quintile	TUNDRA LSOA Quintile	Gaps GCSE Quintile	Et Q
9	0.0	23.0	5.0	4.0	3.0	4.0	3.0	3.0	5.0	
12	0.0	31.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	
13	0.0	32.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	
50	1.0	20.0	5.0	4.0	3.0	4.0	3.0	3.0	5.0	
56	0.0	20.0	5.0	5.0	4.0	4.0	5.0	5.0	4.0	
57	0.0	21.0	5.0	5.0	5.0	4.0	5.0	5.0	5.0	
76	0.0	22.0	4.0	3.0	4.0	4.0	4.0	4.0	4.0	

In [34]:
```
fitur_outlier = [
    'distance to university (km)',
    'Count of Module Area Logins',
    'Total Hours in Module Area'
]

for kolom in fitur_outlier:
    print("\nKolom:", kolom)
    print("Nilai minimum:", df[kolom].min())
    print("Nilai maksimum:", df[kolom].max())
```

```
Kolom: distance to university (km)
Nilai minimum: 0.0
Nilai maksimum: 317.3273631

Kolom: Count of Module Area Logins
Nilai minimum: 1.0
Nilai maksimum: 335.0

Kolom: Total Hours in Module Area
Nilai minimum: 0.03
Nilai maksimum: 175.44
```

Berdasarkan pemeriksaan menggunakan metode IQR, beberapa fitur dapat memiliki nilai yang terdeteksi sebagai outlier. Namun, nilai tersebut tidak langsung dihapus karena masih mungkin menggambarkan kondisi mahasiswa yang sebenarnya. Oleh karena itu, outlier dipertahankan selama tidak terbukti sebagai kesalahan atau nilai yang tidak valid.

In [35]:
```
#Memeriksa nilai tidak valid
kolom_periksa = [
    'Age',
```

```
    'distance to university (km)',
    'Count of Module Area Logins',
    'Total Hours in Module Area',
    '# of presence',
    '# of Absence',
    'Percent Attended'
]

for kolom in kolom_periksa:
    print(kolom, ":", (df[kolom] < 0).sum())
```

```
Age : 0
distance to university (km) : 0
Count of Module Area Logins : 0
Total Hours in Module Area : 0
# of presence : 0
# of Absence : 0
Percent Attended : 0
```

Kesimpulan EDA dan Data Processing

Berdasarkan hasil eksplorasi dan pengolahan data, dataset telah diperiksa dari segi jumlah data kosong, nilai yang hilang, duplikasi, distribusi fitur, hubungan antarfitur, serta keberadaan outlier dan nilai tidak valid.Penanganan dilakukan dengan mempertimbangkan karakteristik setiap fitur agar informasi penting dalam dataset tetap dipertahankan. Visualisasi digunakan untuk memahami pola data dan melihat perbedaan karakteristik mahasiswa berdasarkan status kelulusan.Hasil dari tahap ini menjadi dasar untuk menentukan fitur dan mempersiapkan data sebelum dilakukan pemodelan machine learning.

In [36]:
```
#Persiapan data
```

In [37]:
```
df = df.drop(columns=['TUNDRA LSOA Quintile'])
```

Penghapusan Fitur Redundan

Fitur TUNDRA MSOA Quintile dan TUNDRA LSOA Quintile memiliki nilai yang identik. Oleh karena itu, salah satu fitur dihapus untuk menghindari penggunaan informasi yang sama secara berulang.

In [38]:
```
#memisahkan fitur dan label
X = df.drop(columns=['label (fail=1, pass=0)'])
y = df['label (fail=1, pass=0)']

print("Bentuk fitur:", X.shape)
print("Bentuk label:", y.shape)
```

```
Bentuk fitur: (80, 18)
Bentuk label: (80,)
```

In [39]:
```
#Membagi data training dan testing
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
```

```
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Jumlah data training:", X_train.shape)
print("Jumlah data testing:", X_test.shape)
```

```
Jumlah data training: (64, 18)
Jumlah data testing: (16, 18)
```

In [40]:
```
#memeriksa distribusi
print("Distribusi label data training:")
print(y_train.value_counts())

print("\nDistribusi label data testing:")
print(y_test.value_counts())
```

```
Distribusi label data training:
label (fail=1, pass=0)
0.0    54
1.0    10
Name: count, dtype: int64

Distribusi label data testing:
label (fail=1, pass=0)
0.0    14
1.0     2
Name: count, dtype: int64
```

In [41]:
```
#melakukan scaling fitur
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Bentuk data training setelah scaling:", X_train_scaled.shape)
print("Bentuk data testing setelah scaling:", X_test_scaled.shape)
```

```
Bentuk data training setelah scaling: (64, 18)
Bentuk data testing setelah scaling: (16, 18)
```

StandardScaler digunakan untuk menyamakan skala fitur numerik agar memiliki rata-rata mendekati 0 dan standar deviasi mendekati 1. Scaler hanya dilatih menggunakan data training, kemudian diterapkan pada data testing untuk menghindari kebocoran informasi.

Penanganan Ketidakseimbangan Kelas: Dataset memiliki ketidakseimbangan kelas, yaitu jumlah mahasiswa lulus lebih banyak dibandingkan mahasiswa gagal. Untuk mengatasi hal tersebut, digunakan metode class_weight='balanced' pada model yang mendukungnya. Metode ini membantu model memberikan perhatian lebih pada kelas minoritas tanpa menambah atau mengubah data testing.

In [42]:
```
#Modeling
```

In [43]:
```
#Import algoritma Machine Learning
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
```

In [44]:
```
#Membuat model Logistic Regression
model_lr = LogisticRegression(
    random_state=42,
    class_weight='balanced',
    max_iter=1000
)

model_lr.fit(X_train_scaled, y_train)
```

Out[44]:

▾ LogisticRegression

▸ Parameters

▸ Fitted attributes

In [45]:
```
#Membuat model Random Forest
model_rf = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    class_weight='balanced'
)

model_rf.fit(X_train, y_train)
```

Out[45]:

▾ RandomForestClassifier

▸ Parameters

▸ Fitted attributes

In [46]:
```
#Melakukan prediksi pada data testing
y_pred_lr = model_lr.predict(X_test_scaled)
y_pred_rf = model_rf.predict(X_test)
```

In [47]:
```
#Membandingkan hasil awal kedua model
from sklearn.metrics import accuracy_score

print("Akurasi Logistic Regression:",
      accuracy_score(y_test, y_pred_lr))

print("Akurasi Random Forest:",
      accuracy_score(y_test, y_pred_rf))
```

```
Akurasi Logistic Regression: 0.75
Akurasi Random Forest: 0.9375
```

In [48]:
```
#Cross Validation dan Hyperparameter Tuning
#Import library
from sklearn.pipeline import Pipeline
```

```
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
```

In [49]:
```
#membuat pipeline
pipeline_lr = Pipeline([
    ('scaler', StandardScaler()),
    ('model', LogisticRegression(
        class_weight='balanced',
        random_state=42,
        max_iter=1000
    ))
])
```

Pipeline menggabungkan scaling dan Logistic Regression. Dengan demikian, scaling dilakukan di dalam setiap proses validasi sehingga tidak terjadi kebocoran data (data leakage).

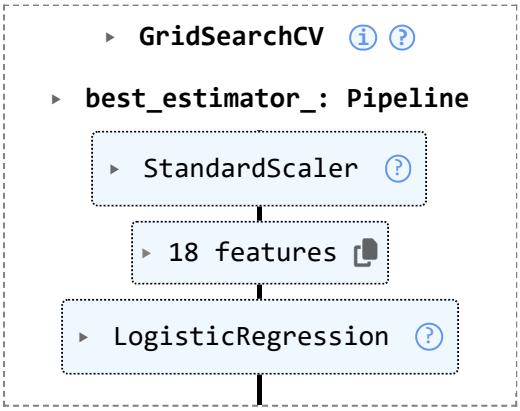
In [50]:
```
#Menentukan parameter yang akan diuji
parameter_grid = {
    'model__C': [0.01, 0.1, 1, 10, 100]
}
```

In [51]:
```
#Melakukan Grid Search
cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

grid_search = GridSearchCV(
    estimator=pipeline_lr,
    param_grid=parameter_grid,
    cv=cv,
    scoring='f1',
    n_jobs=-1
)

grid_search.fit(X_train, y_train)
```

Out[51]:



Menggunakan 5-Fold Stratified Cross Validation dan F1-score sebagai dasar pemilihan parameter. F1-score dipilih karena kita perlu memperhatikan kemampuan model mengenali mahasiswa yang gagal (label 1), bukan hanya mengejar akurasi.

In [52]:
```
#Melihat hasil tuning
print("Parameter terbaik:", grid_search.best_params_)
print("F1-score terbaik:", grid_search.best_score_)
```

```
Parameter terbaik: {'model__C': 100}
F1-score terbaik: 0.7133333333333333
```

In [53]:
```
#Menggunakan model terbaik
best_model = grid_search.best_estimator_

y_pred_lr_tuned = best_model.predict(X_test)
```

In [54]:
```
#Membandingkan performa model
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_sc
import pandas as pd

hasil_model = pd.DataFrame({
    'Model': ['Logistic Regression', 'Random Forest'],
    'Accuracy': [
        accuracy_score(y_test, y_pred_lr_tuned),
        accuracy_score(y_test, y_pred_rf)
    ],
    'Precision': [
        precision_score(y_test, y_pred_lr_tuned, zero_division=0),
        precision_score(y_test, y_pred_rf, zero_division=0)
    ],
    'Recall': [
        recall_score(y_test, y_pred_lr_tuned, zero_division=0),
        recall_score(y_test, y_pred_rf, zero_division=0)
    ],
    'F1-Score': [
        f1_score(y_test, y_pred_lr_tuned, zero_division=0),
        f1_score(y_test, y_pred_rf, zero_division=0)
    ]
})

hasil_model
```

Out[54]:

	Model	Accuracy	Precision	Recall	F1-Score
0	Logistic Regression	0.7500	0.25	0.5	0.333333
1	Random Forest	0.9375	1.00	0.5	0.666667

Karena Logistic Regression sudah melalui proses tuning, kita gunakan model terbaiknya (best_model) untuk dibandingkan dengan Random Forest (model_rf).

Menentukan model terbaik: Perbandingan model dilakukan menggunakan nilai Accuracy, Precision, Recall, dan F1-Score. Pemilihan model terbaik tidak hanya didasarkan pada Accuracy, tetapi juga mempertimbangkan Recall dan F1-Score untuk kelas mahasiswa gagal (label 1). Model dengan kemampuan lebih baik dalam mengenali mahasiswa yang berisiko gagal akan menjadi pertimbangan utama.

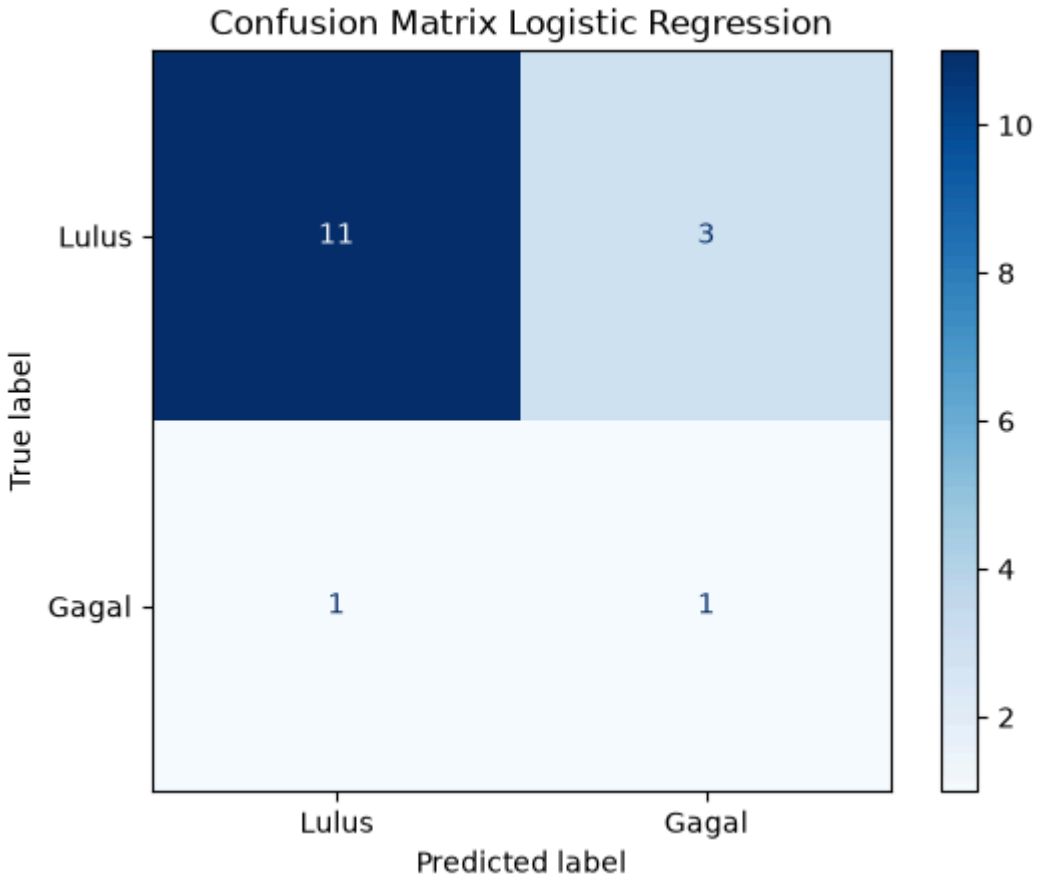
In [55]:
```
#evaluasi model
```

In [56]:
```
#Membuat Confusion Matrix
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# Confusion Matrix Logistic Regression
cm_lr = confusion_matrix(y_test, y_pred_lr_tuned)

disp_lr = ConfusionMatrixDisplay(
    confusion_matrix=cm_lr,
    display_labels=['Lulus', 'Gagal']
)

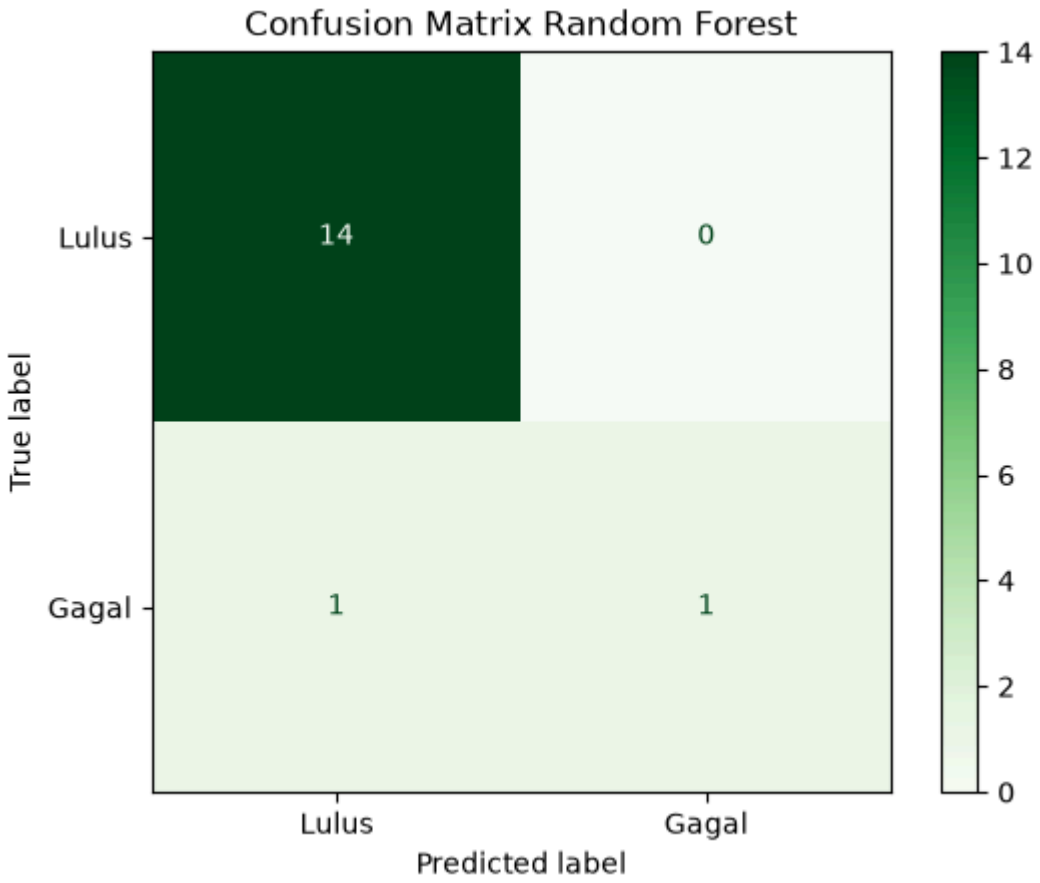
disp_lr.plot(cmap='Blues')
plt.title('Confusion Matrix Logistic Regression')
plt.show()
```



In [57]:
```
#Confusion Matrix untuk Random Forest:
cm_rf = confusion_matrix(y_test, y_pred_rf)

disp_rf = ConfusionMatrixDisplay(
    confusion_matrix=cm_rf,
    display_labels=['Lulus', 'Gagal']
)

disp_rf.plot(cmap='Greens')
plt.title('Confusion Matrix Random Forest')
plt.show()
```



In [58]:
```
#Membuat Classification Report
from sklearn.metrics import classification_report

print("Logistic Regression:")
print(classification_report(
    y_test,
    y_pred_lr_tuned,
    target_names=['Lulus', 'Gagal'],
    zero_division=0
))

print("Random Forest:")
print(classification_report(
    y_test,
    y_pred_rf,
    target_names=['Lulus', 'Gagal'],
    zero_division=0
))
```

```
Logistic Regression:
              precision    recall  f1-score   support

       Lulus       0.92      0.79      0.85        14
       Gagal       0.25      0.50      0.33         2

    accuracy                           0.75        16
   macro avg       0.58      0.64      0.59        16
weighted avg       0.83      0.75      0.78        16

Random Forest:
              precision    recall  f1-score   support

       Lulus       0.93      1.00      0.97        14
       Gagal       1.00      0.50      0.67         2

    accuracy                           0.94        16
   macro avg       0.97      0.75      0.82        16
weighted avg       0.94      0.94      0.93        16
```

In [59]:
```
#Menghitung metrik evaluasi
hasil_evaluasi = pd.DataFrame({
    'Model': ['Logistic Regression', 'Random Forest'],
    'Accuracy': [
        accuracy_score(y_test, y_pred_lr_tuned),
        accuracy_score(y_test, y_pred_rf)
    ],
    'Precision': [
        precision_score(y_test, y_pred_lr_tuned, zero_division=0),
        precision_score(y_test, y_pred_rf, zero_division=0)
    ],
    'Recall': [
        recall_score(y_test, y_pred_lr_tuned, zero_division=0),
        recall_score(y_test, y_pred_rf, zero_division=0)
    ],
    'F1-Score': [
        f1_score(y_test, y_pred_lr_tuned, zero_division=0),
        f1_score(y_test, y_pred_rf, zero_division=0)
    ]
})

hasil_evaluasi
```

Out[59]:

	Model	Accuracy	Precision	Recall	F1-Score
0	Logistic Regression	0.7500	0.25	0.5	0.333333
1	Random Forest	0.9375	1.00	0.5	0.666667

In [60]:
```
#Analisis Feature Importance
importance = model_rf.feature_importances_

feature_importance = pd.DataFrame({
    'Fitur': X_train.columns,
    'Importance': importance
})

feature_importance = feature_importance.sort_values(
```

```
    by='Importance',
    ascending=False
)

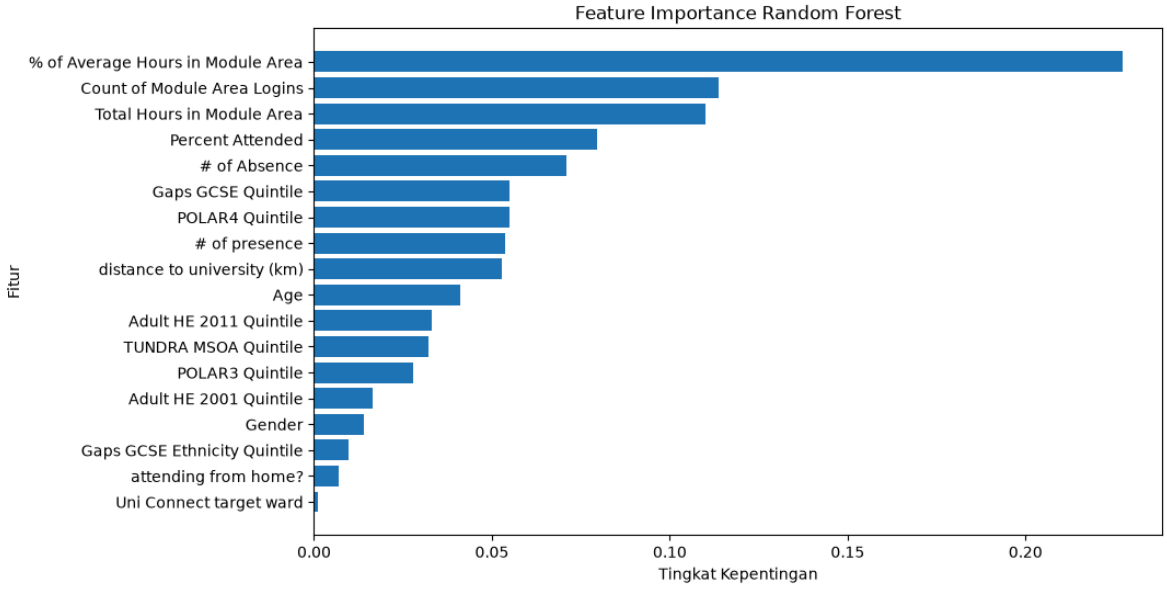
print(feature_importance)
```

	Fitur	Importance
14	% of Average Hours in Module Area	0.227168
12	Count of Module Area Logins	0.113844
13	Total Hours in Module Area	0.109983
17	Percent Attended	0.079678
16	# of Absence	0.070855
7	Gaps GCSE Quintile	0.055088
2	POLAR4 Quintile	0.055004
15	# of presence	0.053663
11	distance to university (km)	0.052665
1	Age	0.040941
5	Adult HE 2011 Quintile	0.032964
6	TUNDRA MSOA Quintile	0.032231
3	POLAR3 Quintile	0.027853
4	Adult HE 2001 Quintile	0.016475
0	Gender	0.014109
8	Gaps GCSE Ethnicity Quintile	0.009620
10	attending from home?	0.006894
9	Uni Connect target ward	0.000966

In [61]:
```
#menampilkan grafik
plt.figure(figsize=(10, 6))

plt.barh(
    feature_importance['Fitur'],
    feature_importance['Importance']
)

plt.xlabel('Tingkat Kepentingan')
plt.ylabel('Fitur')
plt.title('Feature Importance Random Forest')
plt.gca().invert_yaxis()
plt.show()
```



Berdasarkan hasil Feature Importance pada model Random Forest, fitur "% of Average Hours in Module Area" memiliki tingkat kepentingan paling tinggi dalam proses prediksi.

Selanjutnya, fitur "Count of Module Area Logins" dan "Total Hours in Module Area" juga menunjukkan kontribusi yang cukup besar. Sementara itu, fitur "Gender", "Gaps GCSE Ethnicity Quintile", dan "Uni Connect target ward" memiliki tingkat kepentingan relatif rendah.Hasil ini menunjukkan bahwa fitur yang berkaitan dengan aktivitas dan keterlibatan mahasiswa dalam pembelajaran memiliki kontribusi besar terhadap prediksi model. Namun, Feature Importance tidak menunjukkan hubungan sebab-akibat.

In [62]:
```
#Kesimpulan evaluasi
```

Berdasarkan hasil evaluasi, kedua model memiliki performa yang berbeda. Logistic Regression memperoleh Accuracy sebesar 75%, Precision sebesar 25%, Recall sebesar 50%, dan F1-Score sebesar 33,33%. Sementara itu, Random Forest memperoleh Accuracy sebesar 93,75%, Precision sebesar 100%, Recall sebesar 50%, dan F1-Score sebesar 66,67%. Hasil evaluasi menunjukkan bahwa Random Forest memiliki nilai Accuracy, Precision, dan F1-Score lebih tinggi dibandingkan Logistic Regression. Namun, kedua model memiliki nilai Recall yang sama, yaitu 50%. Hal ini menunjukkan bahwa kedua model masih memiliki keterbatasan dalam mengenali mahasiswa yang benar-benar gagal.

In [64]:
```
#Pemilihan Model Terbaik
```

Berdasarkan hasil evaluasi, Random Forest dipilih sebagai model terbaik karena memiliki nilai Accuracy sebesar 93,75%, Precision sebesar 100%, dan F1-Score sebesar 66,67%. Nilai tersebut lebih tinggi dibandingkan Logistic Regression. Meskipun demikian, Recall Random Forest masih sebesar 50%. Oleh karena itu, model ini masih perlu dikembangkan agar kemampuan dalam mengenali mahasiswa yang berisiko gagal dapat ditingkatkan.

In [65]:
```
#Teks prediksi Data Random
```

In [66]:
```
#Menentapkan model terbaik
best_model = model_rf

print("Model yang digunakan: Random Forest")
```

```
Model yang digunakan: Random Forest
```

In [67]:
```
#memeriksa fitur yang digunakan
print("Daftar fitur:")
print(X_train.columns.tolist())

print("\nJumlah fitur:", X_train.shape[1])
```

```
Daftar fitur:
['Gender', 'Age', 'POLAR4 Quintile', 'POLAR3 Quintile', 'Adult HE 2001 Quintile',
'Adult HE 2011 Quintile', 'TUNDRA MSOA Quintile', 'Gaps GCSE Quintile', 'Gaps GCS
E Ethnicity Quintile', 'Uni Connect target ward', 'attending from home?', 'distan
ce to university (km)', 'Count of Module Area Logins', 'Total Hours in Module Are
a', '% of Average Hours in Module Area', '# of presence', '# of Absence', 'Percen
t Attended']

Jumlah fitur: 18
```

In [72]:
```
#membuat 5 data mahasiswa baru
import numpy as np

np.random.seed(42)

data_baru = pd.DataFrame()

for kolom in X_train.columns:
    data_baru[kolom] = np.random.choice(
        X[kolom].values,
        size=5,
        replace=True
    )

data_baru
```

Out[72]:

	Gender	Age	POLAR4 Quintile	POLAR3 Quintile	Adult HE 2001 Quintile	Adult HE 2011 Quintile	TUNDRA MSOA Quintile	Gaps GCSE Quintile	Gaps GCSE Ethnicity Quintile	C
0	0.0	20.0	5.0	1.0	5.0	5.0	1.0	5.0	5.0	
1	0.0	20.0	4.0	5.0	3.0	4.0	4.0	5.0	1.0	
2	0.0	20.0	4.0	4.0	3.0	5.0	3.0	3.0	5.0	
3	0.0	21.0	1.0	2.0	3.0	4.0	4.0	4.0	3.0	
4	0.0	23.0	4.0	1.0	4.0	4.0	1.0	2.0	1.0	

In [73]:
```
#Melakukan prediksi
prediksi = best_model.predict(data_baru)

print("Hasil Prediksi:")
print(prediksi)
```

```
Hasil Prediksi:
[0. 0. 0. 1. 0.]
```

In [74]:
```
#Menampilkan probabilitas prediksi
probabilitas = best_model.predict_proba(data_baru)

print("Probabilitas Prediksi:")
print(probabilitas)
```

```
Probabilitas Prediksi:
[[0.81 0.19]
 [0.57 0.43]
 [0.83 0.17]
 [0.46 0.54]
 [0.75 0.25]]
```

In [75]:
```
#Membuat tabel hasil prediksi
hasil_prediksi = pd.DataFrame({
    'Mahasiswa': ['Mahasiswa 1', 'Mahasiswa 2',
                  'Mahasiswa 3', 'Mahasiswa 4',
                  'Mahasiswa 5'],
```

```
    'Prediksi': [
        'Lulus' if nilai == 0 else 'Gagal'
        for nilai in prediksi
    ],
    'Probabilitas Lulus': probabilitas[:, 0],
    'Probabilitas Gagal': probabilitas[:, 1]
})

hasil_prediksi
```

Out[75]:

	Mahasiswa	Prediksi	Probabilitas Lulus	Probabilitas Gagal
0	Mahasiswa 1	Lulus	0.81	0.19
1	Mahasiswa 2	Lulus	0.57	0.43
2	Mahasiswa 3	Lulus	0.83	0.17
3	Mahasiswa 4	Gagal	0.46	0.54
4	Mahasiswa 5	Lulus	0.75	0.25

Berdasarkan hasil prediksi menggunakan model Random Forest terhadap lima data mahasiswa baru yang dibuat secara acak, diperoleh empat mahasiswa yang diprediksi lulus dan satu mahasiswa yang diprediksi gagal. Mahasiswa 1 diprediksi lulus dengan probabilitas 81%, mahasiswa 2 sebesar 57%, mahasiswa 3 sebesar 83%, dan mahasiswa 5 sebesar 75%. Sementara itu, mahasiswa 4 diprediksi gagal dengan probabilitas sebesar 54%. Hasil tersebut menunjukkan bahwa model dapat memprediksi status kelulusan mahasiswa berdasarkan fitur yang tersedia serta memberikan probabilitas untuk setiap kelas. Namun, hasil prediksi merupakan estimasi model dan bukan kepastian mengenai kelulusan mahasiswa yang sebenarnya.

In [76]:
```
#Kesimpulan Random Data Prediction
```

Berdasarkan pengujian terhadap lima data mahasiswa baru yang dibuat secara acak menggunakan NumPy, model Random Forest berhasil menghasilkan prediksi status kelulusan beserta probabilitasnya. Dari lima data yang diuji, empat mahasiswa diprediksi lulus dan satu mahasiswa diprediksi gagal. Dengan demikian, model dapat digunakan untuk memperkirakan status kelulusan berdasarkan fitur-fitur yang tersedia. Meskipun begitu, hasil prediksi masih memiliki keterbatasan sehingga tidak dapat dijadikan sebagai kepastian hasil akademik mahasiswa.

In [ ]:
```

```